

TARGET SHIP DATE: 2026-06-01

Blossomberg Terminal by Finance_Bros

Roster: Haowen Xiao (PM), Robert Chen, James Sun, Cody Wong

Description:

We aim to create a stock-trading tool called the Blossomberg Terminal. As you can tell, it will be inspired by the Bloomberg L.P terminal that stock traders use on a daily basis(the real thing cost \$25,000). Crazy... Am I right?

How will it be used(generic features)):

- **Access to live stock data**
- **Viewable graphs of these stocks**
- **Dynamic comparisons with our ML models**
- **Interaction with portfolios and data saving**

TLDR: Honestly, to say we'll recreate the "bloomberg terminal" is def overhyped, but we will be doing our greatest with the limited time we got.

Flask App:

init.py (main app + routes)

Creates Flask app, configures sessions, handles routing, and connects backend logic to frontend templates.

/register

Adds new users to database, checks if usernames already exist, hashes passwords, creates sessions, and redirects users to dashboard.

/login

Verifies login credentials, creates sessions, and redirects users to dashboard.

/logout

Clears session data and redirects users back to login page.

/dashboard

Main terminal dashboard displaying stock watchlists, indexed growth charts, comparison graphs, volume charts, and market overview panels while allowing users to search stock tickers, compare equities, and open detailed stock pages.

/stock_viewer

Displays individual stock profiles or comparisons between selected stocks including candlestick charts, historical performance, moving averages, and volatility metrics.

/forecast

Displays machine learning predictions, regression analysis, forecasted trends, confidence intervals, and prediction graphs using models such as Linear Regression, Random Forest Regression, and Logistic Regression with possible future PyTorch LSTM forecasting support.

/portfolio

Allows users to save stocks, create watchlists, simulate portfolios, and track performance over time.

/ai_analysis

Uses [OpenRouter](#) to generate AI stock summaries, market explanations, trendanalysis, and company comparisons using prompts like “Why did NVDA rise today?” or “Compare MSFT and AAPL.”

Database:

data.py

Connects to SQLite database and handles table creation, stock data management, user management, portfolio/watchlist storage, and cached API data.

Database Tables:

USERS			
INTEGER	id	PK	Unique id
TEXT	username		
TEXT	password		

STOCKS			
INTEGER	id	PK	Unique, Auto-increment
TEXT	ticker		Few letters representing a stock
TEXT	company_name		

users

Stores usernames, hashed passwords, and user account information.

Stocks

STOCK_DATA			
INTEGER	stock_id	FK	
TIMESTAMP	date		
FLOAT	open_price		Stock price upon market open
FLOAT	close_price		Stock price upon market close
FLOAT	returns		Gains/losses between consecutive days
INTEGER	volume		Number of stock multiplied by price of stock

WATCHLISTS			
INTEGER	id	PK	Watchlist row
INTEGER	user_id	FK	Linked user
INTEGER	stock_id	FK	Linked stock

Stores stock tickers, company names, sectors, and stock metadata for equities such as AAPL, NVDA, TSLA, META, AMZN, and MSFT.

stock_data

Stores historical market data including open prices, close prices, highs, lows, dates, and trading volume used for charting, analytics, and ML forecasting.

watchlists

Stores user-selected stocks and portfolio/watchlist information linked to accounts.

Database Organization:

The database is intentionally kept relatively simple for the MVP and mainly exists to store stock prices, historical trading data, watchlists, portfolio information, cached API data, and values needed for graphs, analytics, and machine learning calculations.

Core Components:

- [Chart.js](#)
- [Scikit-learn](#) ML models (Classification + Regression)
- [PyTorch](#) (future possibility)
- [OpenRouter](#) for AI analysis
- Google Colab integration through cloud APIs
- Flask backend with SQLite database

Front-End Framework:

TailwindCSS will be used for responsive layouts and dashboard styling while JavaScript will handle chart updates, stock searching, API communication, filtering, comparisons, and interactive frontend behavior.

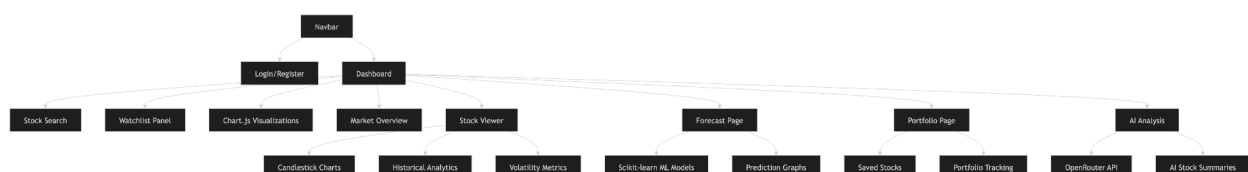
APIs:

- [Yahoo Finance API Docs](#) for historical stock data and market information
- [Alpaca Markets](#) for optional live market data
- [Playwright](#) for automation and scraping
- [Apify](#) for data collection pipelines
- [Crawlee](#) for web crawling workflows

Data Visualization Library:

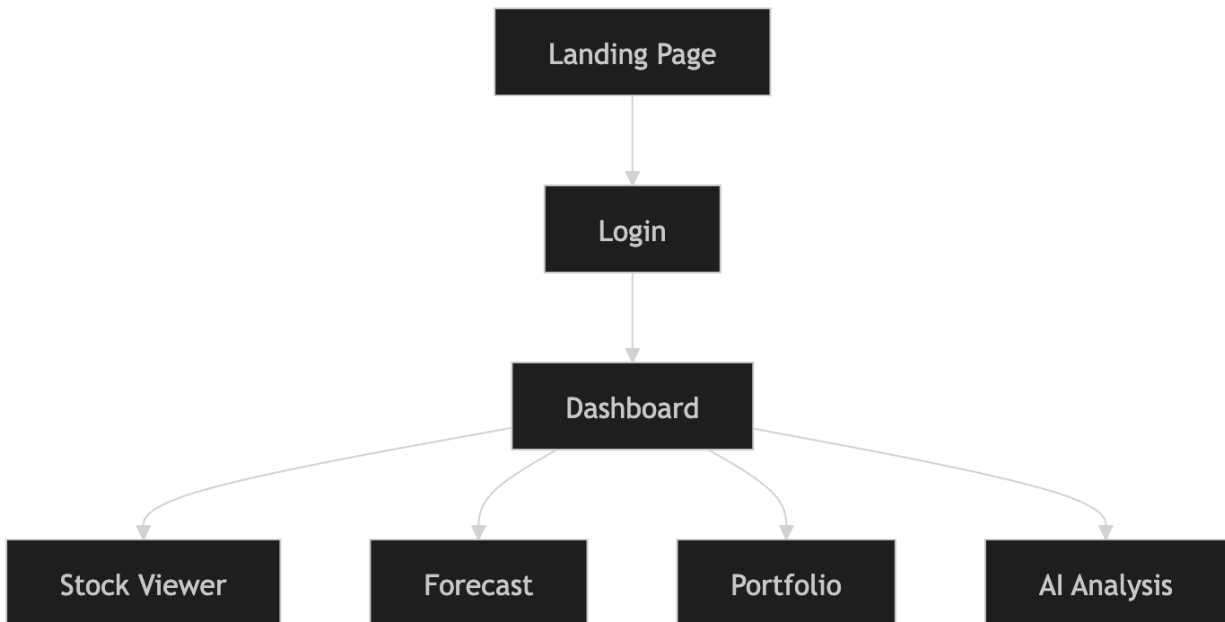
Primary graphing library will be [Chart.js](#) for stock charts, prediction graphs, comparison visualizations, moving averages, and volume charts.

Component Map:

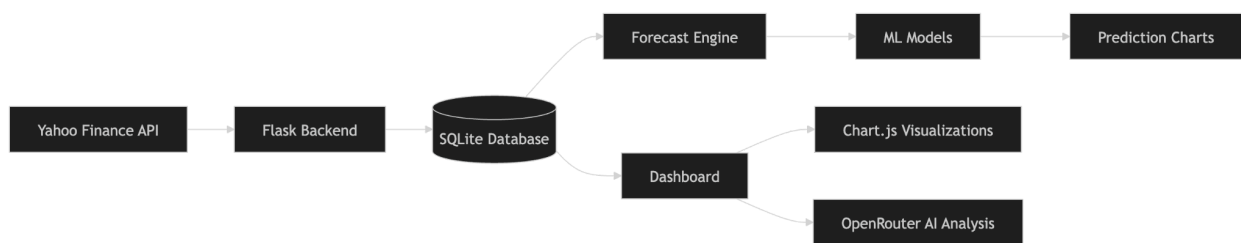


Navbar, Login/Register system, Dashboard Panel, Stock Viewer, Forecast Page, Portfolio Tracker, AI Analysis Panel, Chart Components, Stock Selector Dropdowns, Watchlist System, and ML Prediction Visualizations.

Site Map:



Data Flow:



Task Breakdown:

Haowen Xiao

Project management, deployment, frontend structure, dashboard UI, and overall architecture.

Robert Chen

Flask backend, authentication system, routing, sessions, dashboard logic, and portfolio pages.

James Sun

Machine learning systems, forecasting models, AI integrations, regression analysis, and prediction pipelines.

Cody Wong

SQLite database organization, stock datasets, backend data handling, API integration, watchlist systems, and analytics infrastructure.